Project report

Introduction

In this project, we created a simulation that works like a simple operating system scheduler. The main goal was to understand how the CPU manages several jobs at the same time and how resources like memory and devices are shared.
We used the round robin (RR) scheduling algorithm, which gives every job a fair share of CPU time. It is one of the most common and easy to understand scheduling techniques in operating systems

Our project shows how each process moves through the system from being added, to waiting in a queue, to finally finishing its execution. The simulation prints all system details step by step so we can clearly see what happens inside the scheduler.

This project helped us connect what we studied in lectures with real programming. By building and testing it, we gained a better understanding of how an operating system handles multiple processes, prevents starvation, and maintains fairness

Project Description

The simulator we built reads commands from a file called input.txt. Each command represents a system event or job action. The program then reacts to these commands and updates the system accordingly

The main types of commands are:

-C (Configuration): sets up the total amount of memory and devices available in the system

-A (Arrival): adds a new job into the system with information such as memory needed, devices needed, burst time, and priority

-D (Display): prints the current system state at that specific time

When the program runs, it creates an output.txt file that shows the full system status. This file includes the total and available memory, devices, and a list of jobs in each queue:

-Ready List: jobs that are waiting for CPU time

-Hold List 1: jobs with higher priority waiting for resources.

-Hold List 2: jobs with lower priority waiting for resources.

-Finished Jobs List: jobs that have completed execution, including their turnaround and weighted turnaround times.

This makes it easy to see how the system changes after every command

Implementation

We used **Java** to build this project because it supports object-oriented programming and is easy to test.
The program is divided into several classes, and each one has a clear job, similar to how real operating system components work.

Class Name	Why did we need it
Main.java	Starts the simulation and controls input and output.
SimulationController.java	Reads the input file and controls the order of events
PrManager.java	Manages queues, CPU scheduling, and job movement
ProcessControlBlock.java	Holds all information for a single job (arrival time, burst time, etc.)
DRoundRobinScheduler.java	Implements Dynamic Round Robin using changing time quantum
SRoundRobinScheduler.java	Implements Static Round Robin using a fixed time quantum
OtherKerServices.java	Manages memory and device allocation
QueueQ.java	Handles basic queue operations (add, remove, check if empty)

The main idea behind the code is to simulate real operating system behavior step by step. When a job arrives, the system checks if there are enough resources.

-If yes, it moves the job to the Ready Queue.

-If not, it goes to one of the Hold Lists until memory or devices become available.

The CPU then takes one job from the ready queue, runs it for a specific time (based on the time quantum), and either finishes it or returns it to the queue

We created two types of scheduling:

-Static Round Robin (SRR)**:** Every job gets a fixed time slice (for example, 13 units)

-Dynamic Round Robin (DRR)**:** The time slice changes depending on the average remaining burst time of the ready jobs

This design makes the simulator flexible and realistic

Progress and Testing

When we started working on this project, our first goal was honestly just to make something that actually runs. So we began with the very basic part, which was reading the input file line by line and printing the system configuration. After that worked, we slowly added more pieces, like handling job arrivals and putting the jobs in the correct queues (ready, hold 1, hold 2).

Once the main structure was working, we added the scheduling part (SRR and DRR). This took some tries because sometimes the CPU ran the same job again by mistake or the memory wasn't updated correctly. We had to adjust the dispatching and the internal event function so everything updates in the right time.

For testing, we tried different kinds of input files:

-Normal test: few jobs, all of them can run normally

-Heavy test: many jobs so we can check how the queues fill up

-Edge cases: jobs that request too much memory (so they get rejected) or high priority jobs arriving at the same time

Each test helped us notice small bugs. For example, sometimes a job wasn't removed from the queue, or the memory didn't get released when the job finished. After fixing these problems and running the tests again, the simulator started giving the correct final results for all the inputs we tried.

In the end, the output became clear and organized. It shows the ready list, the hold lists, memory/device status, and all the finished jobs with their completion and turnaround times. The program calculates everything automatically once the simulation reaches the display command.

Result and Comparison

The following table shows the results for both scheduling algorithms

Job ID	Arrival Time	Completion (SRR)	Completion (DRR)	Turnaround (SRR)	Turnaround (DRR)	Waiting (SRR)	Waiting (DRR)
1	0.00	5.00	5.00	5.00	5.00	0.00	0.00
2	0.00	10.00	10.00	10.00	10.00	5.00	5.00
3	0.00	15.00	15.00	15.00	15.00	10.00	10.00
4	5.00	20.00	20.00	15.00	15.00	10.00	10.00
5	—	—	—	—	—	—	—
6	5.00	25.00	25.00	20.00	20.00	15.00	15.00

From the table we made, we can see that both Static Round Robin (SRR) and Dynamic Round Robin (DRR) basically gave the same exact results for all the jobs that actually ran (Jobs 1, 2, 3, 4 and 6). The completion time, turnaround time, and waiting time are matching in both cases and there is no real different between the two schedulers in this input.

The main reason for that is because all the jobs have the same CPU burst (all of them have R = 5), so the dynamic quantum the DRR calculates ends up being almost the same every time. So DRR didn't really show any special advantage here because the data itself is too simple.

Job 5 never ran in both schedulers because it asked for too much memory (M = 241) while the system only has 100, so it gets rejected straight away.

Overall, for this specific input, both SRR and DRR behave almost identical and the system produces the same finishing results. Maybe with a more complex input the difference would show more clearly, but in this case they're equal.

Conclusion:

In conclusion, building this project helped us understand how an operating system style scheduler works in real life. At the beginning it was confusing, but step by step we managed to connect everything together, like memory management, queues, and the two different round robin schedulers. After testing with many input files, the simulator started giving stable and correct outputs.

We also learned how jobs move between the ready list, hold lists, and finally finish, depending on the system resources. Comparing SRR and DRR also showed us how different time-quantum strategies can change the results. Overall, the project was useful because it made us practice both Java coding and OS concepts at the same time.

Example from output.txt.

```
CONFIG at 2.00: mem=100 devices=1 scheduler=DynamicRR
Q = Dynamic

-------------------------------------------------------
System Status:
-------------------------------------------------------
        Time: 999999.00
  Total Memory: 100
 Avail. Memory: 100
 Total Devices: 1
Avail. Devices: 1

Jobs in Ready List
-------------------------------------------------------
  EMPTY

Jobs in Long Job List
-------------------------------------------------------
  EMPTY

Jobs in Hold List 1
-------------------------------------------------------
  EMPTY

Jobs in Hold List 2
-------------------------------------------------------
  EMPTY


Finished Jobs (detailed)
-------------------------------------------------------
  Job    ArrivalTime     CompleteTime     TurnaroundTime    WaitingTime
-------------------------------------------------------------------------
  1      0.00            5.00             5.00              0.00
  2      0.00            10.00            10.00             5.00
  3      0.00            15.00            15.00             10.00
  4      5.00            20.00            15.00             10.00
  6      5.00            25.00            20.00             15.00


--- Simulation finished at time 999999.0 ---
```

```
CONFIG at 2.00: mem=100 devices=1 scheduler=StaticRR
Q = 13

-------------------------------------------------------
System Status:
-------------------------------------------------------
        Time: 999999.00
  Total Memory: 100
 Avail. Memory: 100
 Total Devices: 1
Avail. Devices: 1

Jobs in Ready List
-------------------------------------------------------
  EMPTY

Jobs in Long Job List
-------------------------------------------------------
  EMPTY

Jobs in Hold List 1
-------------------------------------------------------
  EMPTY

Jobs in Hold List 2
-------------------------------------------------------
  EMPTY


Finished Jobs (detailed)
-------------------------------------------------------
  Job    ArrivalTime     CompleteTime     TurnaroundTime    WaitingTime
-------------------------------------------------------------------------
  1      0.00            5.00             5.00              0.00
  2      0.00            10.00            10.00             5.00
  3      0.00            15.00            15.00             10.00
  4      5.00            20.00            15.00             10.00
  6      5.00            25.00            20.00             15.00
  1      0.00            5.00             5.00              0.00
  2      0.00            10.00            10.00             5.00
  3      0.00            15.00            15.00             10.00
  4      5.00            20.00            15.00             10.00
  6      5.00            25.00            20.00             15.00


--- Simulation finished at time 999999.0 ---
```